

Sed: searching for any non-space sign is case sensitive

Asked 14 years, 3 months ago Modified 14 years, 3 months ago Viewed 2k times



My goal is to add spaces before and after every mathematical operator in program code. For that reason, I use the following sed command:

1



```
sed
-e 's/\([^ ->]\)\([\+=-\\*]\)/\1 \2/g'
-e 's/\([\+=-\\*]\)\([^ ->]\)/\1 \2/g'"
input > output
```



First line searches for a non-space (or not -> operator) followed by mathematical operator and replaces it with both characters separated by space. The other does the same with characters in opposite order.

However, apart from desired behaviour, which works it also adds spaces between all uppercase letters. I also tested it with option I (case-insensitive) and then all words in file were separated into single characters by space, like this:

```
.....d r a i n c u r r e n t w i t h o u t v e l o c i t y s a t u r a t i o n e f f e c
t
```

I can see no possible relation between expected behaviour and case-sensitivity, would be thankful for any insight on that issue.

sed **whitespace** **case-sensitive**

Share Improve this question Follow

asked May 31, 2012 at 19:38



[Warmonger](#)

23 7

1 What about `i++` and stuff? A sample input and desired output wouldn't hurt. – [Lev Levitsky](#) May 31, 2012 at 19:46

Should those double quotes be in there? When I ran without them I got "Invalid range end", until I changed `\+=-\\*` to `*\\+=-`, whereupon I got no strange behavior. – [Beta](#) May 31, 2012 at 20:12

1 The `[\^ ->]` bit says "match anything that is not in the character range between space and '>'. You can't expect `->` to match a the two-character token you're hoping it does when you put it in a character class... – [twalberg](#) May 31, 2012 at 20:15

1 Answer

Sorted by: Highest score (default)



3



the first thing I've noticed is that you're operating on a character-class range where you probably don't want to do so:

```
[^ ->]
```

The above snippet actually means "match anything that's not between SPACE and GREATER_THAN, unless you escape the minus sign. On an ASCII-based system, for example, this would include any of the following characters also: `!"#$%&'()*+,-./0123456789:;<=`

Also, as far as I know, the POSIX regular expression syntax does not actually offer a way to say "match everything except the sequence "xyz" (with Perl one could use something like `((?!xyz)[a-z]+)` to match any sequence of lowercase characters except those starting with "xyz").

However, here's the way I would go around this if I'd be forced to use `sed`:

```
mjhennig@blackbox:~$ sed \
> -e 's/->/T_PARAM_OBJECT_ACCESS/g' \
> -e 's/\([^\\s]\\)\([+\\-=*]\\)/\1 \2/g' \
> -e 's/\([+\\-=*]\\)\([^\\s]\\)/\1 \2/g' \
> -e 's/T_PARAM_OBJECT_ACCESS/->/g' <<< "alpha->prop+bravo - test=check"
alpha->prop + bravo - test = check
```

As you can see, the missing or maybe unknown-to-me feature is solved by first replacing the occurrences of `->` and converting them back afterwards.

Well, usually this is probably better solved with `awk` or `perl`, though.

Cheers!

Share Improve this answer Follow

answered May 31, 2012 at 20:43



[mjhennig](#)

681 1 5 11

1 Comment ^

Add a comment



[Warmonger](#) Over a year ago

Thank you for your answers. I didn't want to use `->` as a string, but two separate chars. Either way, it was unexpected syntax issue.

▲ 0

Reply



Start asking to get answers

Find the answer to your question by asking.

[Ask question](#)

Explore related questions

`sed`

`whitespace`

`case-sensitive`

See similar questions with these tags.